NLP – NLTK

Matteo Cristani
Computer Science Department
University of Verona - Italy



AGENDA

- Introducing NLTK
- Using NLP techniques in Python
- NLTK and external NLP libraries
- Working with real textual data
- Challenging homeworks

PYTHON AND NLTK BASIC FEATURES



NLTK

- **NLTK — Natural Language Toolkit** is a Python library designed for teaching, experimentation, and prototyping in Natural Language Processing.
- It provides:
  - tools for **tokenization and sentence segmentation**;
  - **stemming and lemmatization**;
  - access to **corpora and lexical resources**;
  - frequency distributions and **n-grams**;
  - basic tools for tagging, parsing, classification and language modelling.

USING NLP TECHNIQUES IN PYTHON



CLASSWORK 1

- Given the following text:
  - On 14 March 2026, Dr. Smith sent 3 reports to the NLP Lab. The meeting started at 10:30 and ended at 12:15. For information, write to nlp-lab@example.org.
- Using Python regular expressions, write patterns to extract:
  - all **dates**;
  - all **times**;
  - all **integer numbers**;
  - the **email address**.
- For each pattern:
  - print all matches;
  - count how many matches were found;
  - briefly explain which part of the regular expression identifies the target structure.

CLASSWORK 2

- Consider the following text:
- Prof. Brown met Dr. Green on 3/9/2026.
  - The next meeting is scheduled for 03-10-2026 at 9:00. Contact john.smith@univr.it or support-team@example.com. The experiment obtained scores of 91.5%, 87%, and 0.93. Version 3.14 of the software was released yesterday.
- Design regular expressions that extract:
  - the different **date formats**;
  - **email addresses**;
  - **percentages**;
  - **decimal numbers**.
- Then inspect the results and answer:
- Does the pattern for decimal numbers also identify 3.14 in Version 3.14?
- Can the same string satisfy more than one pattern?
- Which expressions would require additional context to determine their linguistic function?
- The objective is to observe that **matching a character pattern does not necessarily identify its linguistic meaning**.

CLASSWORK 3

- Consider:
  - Dr. Smith works at the University of Verona. He studies NLP. The experiment used version 3.14. Was it successful? Yes, it was!
- First divide the text into sentences using a simple Python or regular-expression strategy.
- Then use NLTK:
- from nltk.tokenize import sent_tokenize to compare the two outputs.
- Identify:
  - correctly detected sentence boundaries;
  - boundaries incorrectly introduced by the simple method;
  - cases where punctuation does **not** represent the end of a sentence.
- Finally, explain why sentence segmentation cannot in general be reduced to:
  - split whenever we encounter ".", "?" or "!"

CLASSWORK 4

- Consider:
  - Dr. Smith can't attend today's NLP meeting. The state-of-the-art system achieved 97.5%. Visit https://www.example.org for details.
- First tokenize the text using a simple strategy such as:
  - text.split()
- Then use:
  - from nltk.tokenize import word_tokenize
- Compare the two token sequences.
- Focus particularly on:
  - punctuation;
  - contractions such as can't;
  - possessive forms such as today's;
  - hyphenated expressions;
  - numbers and percentages;
  - URLs.
- For every difference, decide whether the two methods are applying a different **tokenization decision** rather than simply producing a right or wrong answer.

NLTK AND EXTERNAL NLP LIBRARIES



CLASSWORK 5: Porter

- Consider the following text:
  - The researchers connected several systems. They are connecting new components and testing the connections. The studies also considered different processing strategies.
- Using NLTK's **Porter Stemmer**:
  - from nltk.stem import PorterStemmer
  - perform the following tasks:
  - tokenize the text into words;
  - apply Porter stemming to every alphabetic token;
  - print each original word together with its stem;
  - identify cases in which different wordforms are reduced to the same stem;
  - identify stems that are **not valid English words**.

WORDNET

- **WordNet** is a large lexical database for English in which words are organized according to their **meanings and semantic relations**.
- Its basic unit is the **synset**: a set of words that express the same concept.
- Example:
  - car, automobile, auto, machine, motorcar
- WordNet also represents relations such as:
  - **synonymy** — words with similar meanings;
  - **antonymy** — opposite meanings;
  - **hypernymy** — more general concepts;
  - **hyponymy** — more specific concepts.
- In NLTK, WordNet is also used as a lexical resource for **lemmatization**:

CLASSWORK 6: Wordnet lemmatization

- Consider the following text:
  - The children were running through the fields. Some mice had eaten the food, while the better trained dogs were watching them.
- Using NLTK's **WordNet Lemmatizer**:
  - from nltk.stem import WordNetLemmatizer
  - perform the following tasks:
  - tokenize the text;
  - lemmatize the words first **without specifying the part of speech**;
  - then lemmatize selected words using the appropriate POS;
  - use porter's
  - compare the three outputs;
  - identify irregular forms that are correctly reduced to their lemma.

CLASSWORK 7: FREQUENCY DISTRIBUTION

- Consider the following text:
  - Natural language processing studies language with computational methods. Language models learn patterns from language data. Computational models can reveal frequent words and recurring structures.
- Using NLTK, perform the following tasks:
  - tokenize the text;
  - convert tokens to lowercase;
  - keep only alphabetic tokens;
  - build a **frequency distribution**;
  - print the 10 most frequent words;
  - report the frequency of the words language, models, and computational;
  - identify the number of tokens and the number of distinct types.
- Use:
  - from nltk import FreqDist

CLASSOWORK 8: CONCORDANCES

- Consider the following text:
  - Language models learn from language data. Modern language models can generate fluent text. The study of language models requires careful analysis of context. Language data may come from many different domains.
- Using NLTK, perform the following tasks:
  - tokenize the text;
  - create an NLTK Text object;
  - display the **concordance** of the word language;
  - display the concordance of the word models;
  - observe the words occurring immediately around each occurrence;
  - compare the contexts in which the same word appears.
- Use:
  - from nltk.text import Text

CLASSWORK 9: COLLOCATIONS

- Consider the following text:
  - Natural language processing is an important field of artificial intelligence. Many natural language processing systems analyse large text collections. Machine learning methods are widely used in natural language processing. Artificial intelligence applications often rely on machine learning.
- Using NLTK, perform the following tasks:
  - tokenize and lowercase the text;
  - keep only alphabetic tokens;
  - identify frequent **bigrams**;
  - use NLTK's collocation tools to extract the most significant bigram collocations;
  - compare simple bigram frequency with an association measure such as **PMI**.
- Useful classes include:
  - from nltk.collocations import BigramCollocationFinder
  - from nltk.metrics import BigramAssocMeasures

CLASSWORK 10: N-GRAM

- Consider the following text:
  - The language model predicts the next word. The model learns patterns from language data. The next word depends on the previous words.
- Using NLTK, perform the following tasks:
  - tokenize and lowercase the text;
  - keep only alphabetic tokens;
  - generate **unigrams, bigrams and trigrams**;
  - print the first 10 bigrams and trigrams;
  - count how many distinct bigrams and trigrams occur;
  - identify repeated n-grams, if any.

CLASSWORK 11: MAXIMUM LIKELIHOOD

- Consider the following text:
  - the cat sat on the mat
    the cat ate the fish
    the dog sat on the mat
- Using NLTK and Python counters, perform the following tasks:
  - tokenize and lowercase the text;
  - generate all **bigrams**;
  - count unigram and bigram frequencies;

CLASSWORK 11: MAXIMUM LIKELIHOOD

- estimate the conditional probability
- $P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$
  - for the following pairs:
  - P(cat | the)
  - P(dog | the)
  - P(sat | cat)
  - P(ate | cat)
  - P(mat | the)
- compare the resulting probabilities;
- identify a plausible unseen bigram, such as:
  - the mouse
  - and compute its probability with maximum-likelihood estimation.
- Then answer:
- Why does an unseen bigram receive probability zero, and what problem does this create for a complete sentence probability?

CLASSWORK 12: LAPLACIAN SMOOTHING

- Starting from the bigram probabilities computed in the previous exercise, apply **Laplace smoothing**:
  - $P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1},w_i)+1}{C(w_{i-1})+|V|}$
- Perform the following tasks:
  - compute the vocabulary size $|\ V\ |$;
- calculate the smoothed probabilities for:
  - P(cat | the)
  - P(dog | the)
  - P(mat | the)
  - P(mouse | the)
- compare the smoothed values with the original maximum-likelihood estimates;
- verify that the unseen bigram the mouse now receives a non-zero probability;
- observe that probabilities of already observed bigrams decrease slightly.

CLASSORK 13: PERPLEXITY

- Assume a **bigram language model**.
- Evaluate the test sequence:
  - the cat sat on the mat
- Perform the following tasks:
  - compute the bigram probabilities for the sentence;
  - multiply them to obtain the sentence probability;
  - compute the perplexity:
    - $PP(W) = P(W)^{-1/N}$
  - where $N$is the number of predicted words;
  - repeat the computation using **Laplace-smoothed probabilities**;
  - compare the two perplexity values.
- Then try a sequence containing an unseen bigram, for example:
  - the mouse sat on the mat
- and observe what happens without smoothing.
- Finally, answer:
- Why does lower perplexity indicate that the language model predicts the test sequence better?

WORKING WITH REAL CORPUSES



Classwork — Exploring a real corpus

- Using the supplied extract—or, preferably, the complete text of *Alice's Adventures in Wonderland*—write a Python program that performs the following analysis:
- **Load and inspect the corpus**
  - read the text from file;
  - tokenize it with NLTK;
  - retain alphabetic tokens and convert them to lowercase;
  - report the number of **tokens** and **types**.
- **Analyse its vocabulary**
  - construct an NLTK FreqDist;
  - display the 20 most frequent words;
  - report the frequencies of alice, rabbit, and queen.

Classwork — Exploring a real corpus

- **Explore words in context**
  - construct an NLTK Text;
  - display concordances for alice and rabbit;
  - briefly describe what information the concordance gives that frequency alone does not.
- **Find collocations**
  - use BigramCollocationFinder;
  - discard bigrams occurring fewer than 3 times;
  - identify the 10 strongest bigram collocations using PMI.
- **Analyse word sequences**
  - generate bigrams and trigrams;
  - identify the 10 most frequent bigrams and trigrams;
  - compare the number of distinct unigrams, bigrams, and trigrams.

Classwork — Exploring a real corpus

- **Interpret the results**
  - Which words characterize the text?
  - Do the collocations reveal expressions or character names that simple frequency counts do not?
  - What happens to repetition as $n$increases from unigram to bigram to trigram?
  - How do these observations relate to the **sparsity problem** discussed in the lecture?

CHALLENGING HOMEWORKS



Homework 1 — Compare two real corpora

- Choose two freely available English corpora that differ clearly in **genre or domain**, for example:
  - literary text vs news;
  - fiction vs scientific/technical text;
  - formal vs informal language.
- For each corpus:
  - load and preprocess the text;
  - compute token and type counts;
  - compute the 20 most frequent words;
  - examine at least two concordances;
  - identify significant bigram collocations;
  - compute the most frequent bigrams and trigrams.

Homework 1 — Compare two real corpora

- Then compare the corpora and answer:
  - Which corpus has the richer observed vocabulary?
  - Which words or collocations characterize each corpus?
  - How different are their most frequent n-grams?
  - Which differences can plausibly be explained by **genre or domain**?
  - How might preprocessing choices affect the comparison?
- **Deliverable:** Python code + a short interpretation of the results.

Homework 2 — Build and test a simple bigram language model

- Choose a freely available corpus and divide it into:
  - a **training portion**;
  - a **test portion**.
- Using the training data:
  - tokenize and normalize the corpus;
  - estimate unigram and bigram frequencies;
  - compute maximum-likelihood bigram probabilities;
  - identify bigrams in the test set that were unseen during training;
  - Compute their proportion.

Homework 2 — Build and test a simple bigram language model

- Implement **Laplace smoothing**;
  - compare unsmoothed and smoothed probabilities for selected bigrams;
  - compute the probability of at least two short test sentences;
  - compare their perplexities.
- Finally, discuss:
  - how corpus size affects the number of unseen bigrams;
  - why sparsity appears even with natural text;
  - what effect smoothing has on observed and unseen events;
  - whether perplexity changes when the test text differs in genre from the training text.